Minimum Spanning Tree

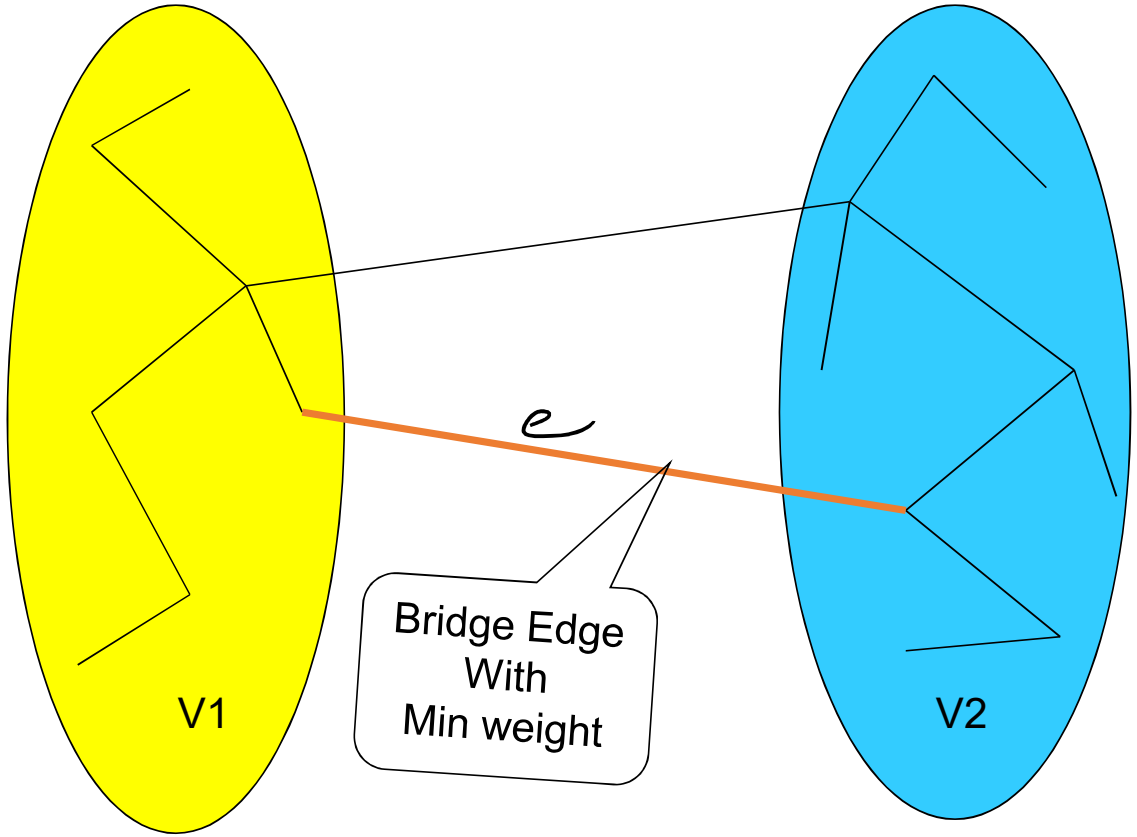
- A *spanning tree* of an undirected graph *G* is a **subgraph of *G*** that is a **tree** containing **all the vertices** of *G*.
- In a weighted graph, the weight of a subgraph is the sum of the weights of the edges in the subgraph.
- A *minimum spanning tree* (MST) for a weighted undirected graph is a spanning tree with minimum weight.

We are interested in:

Finding a tree T that contains all the vertices of a graph G ⇨ ***spanning tree***
and has the least total weight over all such trees ⇨ ***minimum-spanning tree (MST)***

$$w(T) = \sum_{(v,u)\in T} w((v,u))$$

Crucial Fact about MST



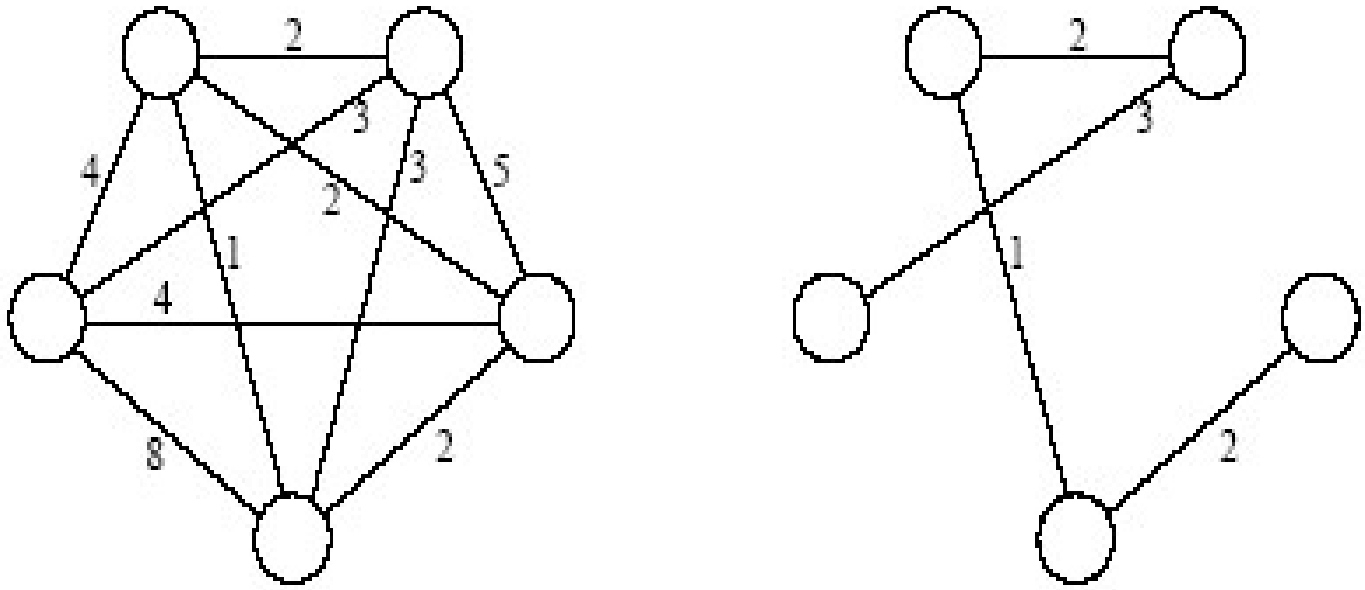
The Crucial Fact about MST

-

The basis of the following algorithms

Proposition: Let $G = (V,E)$ be a weighted graph, and let V_1 and V_2 be two disjoint nonempty sets such that $V = V_1 \cup V_2$. Furthermore, let e be an edge with minimum weight from among those with one vertex in V_1 and the other in V_2. There is a minimum- spanning tree T that has e as one of its edges.

Minimum Spanning Tree



An undirected graph and its minimum spanning tree.

Strategy : Greedy algorithms

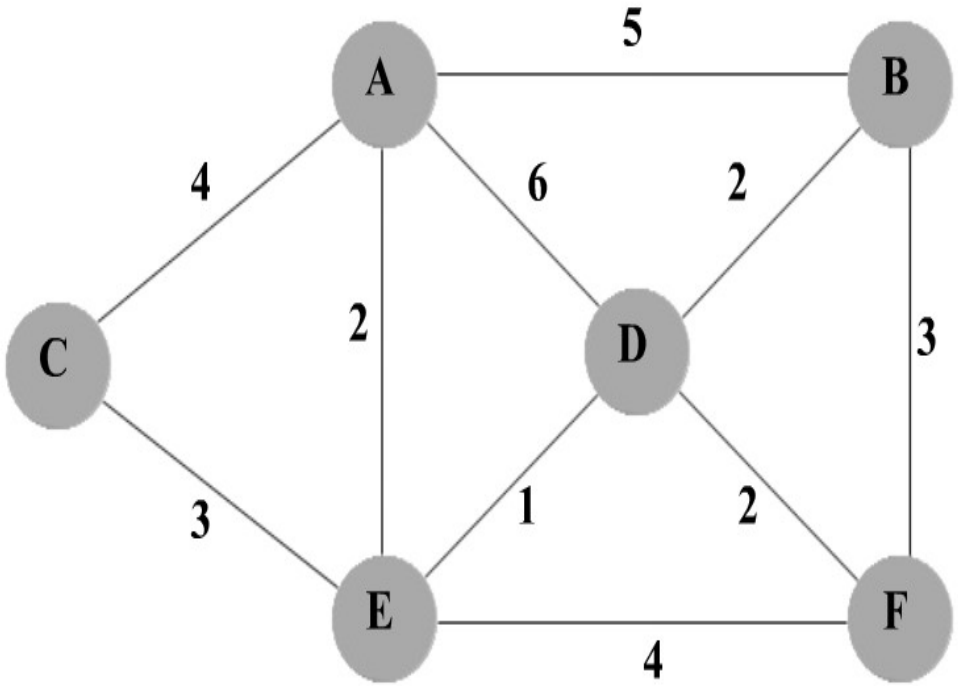
- A *greedy algorithm* always makes the choice that looks best at the moment
  - The hope: a locally optimal choice will lead to a globally optimal solution
  - For **some problems**, it works
- greedy algorithms tend to be easier to code

Prim's Algorithm

- Start with tree T_1 consisting of one (any) vertex and "grow" tree one vertex at a time to produce MST through a series of expanding subtrees T_1, T_2, …, T_n
- On each iteration, construct T_{i+1} from T_i by adding vertex not in T_i that is closest to those already in T_i (this is a "**greedy**" step!)
- Stop when all vertices are included

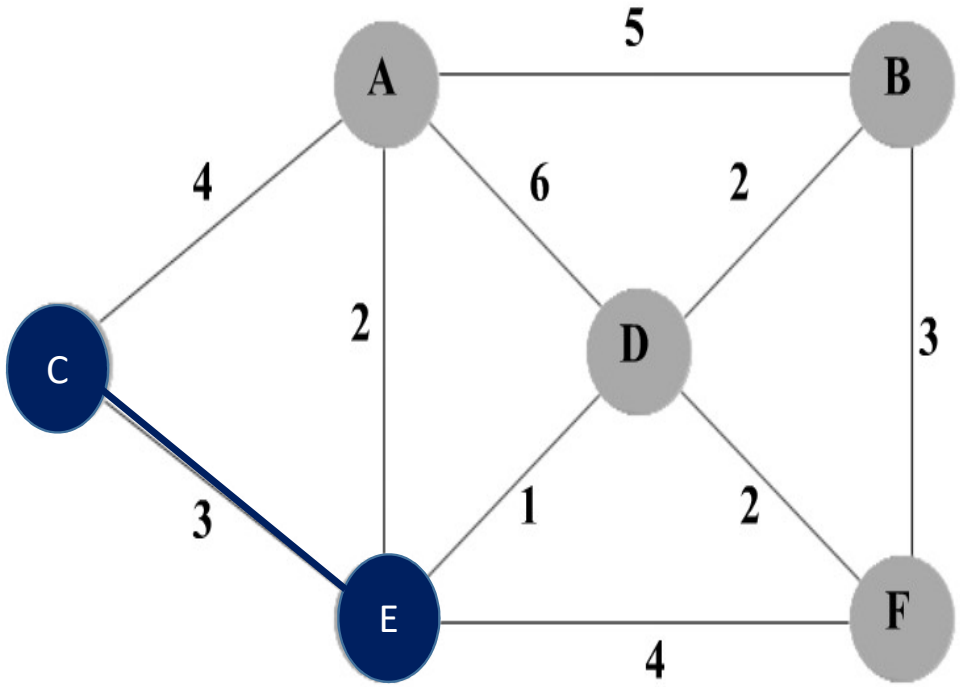
Prim's Algorithm

Node	A	B	C	D	E	F
Disjoint Set	A	B	C	D	E	F



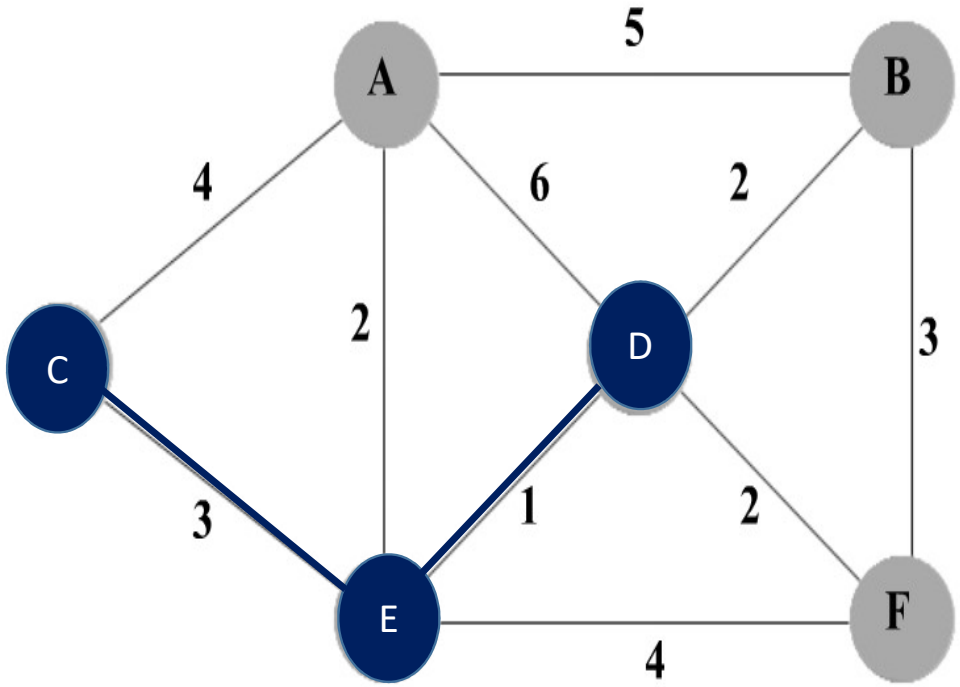
Prim's Algorithm

Node	A	B	C	D	E	F
Disjoint Set	A	B	C	D	E	F



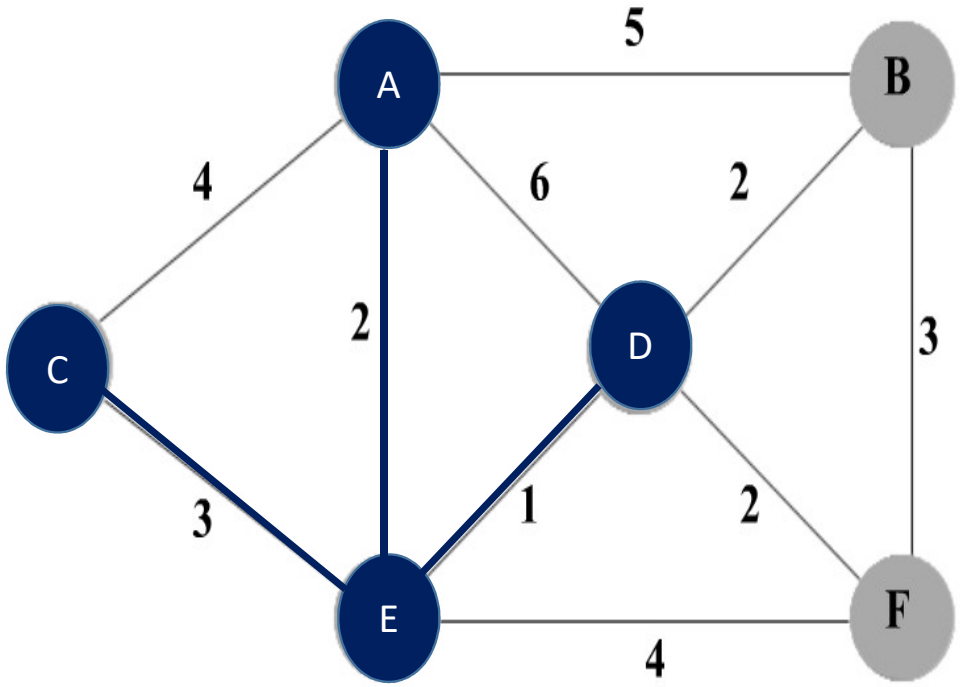
Prim's Algorithm

Node	A	B	C	D	E	F
Disjoint Set	A	B	C	D	E	F



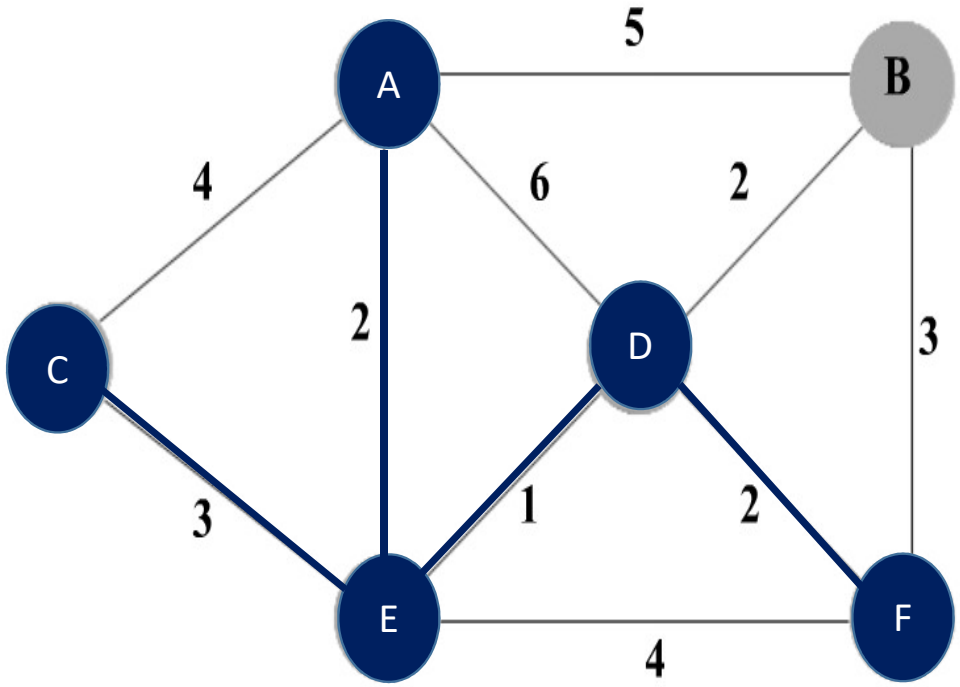
Prim's Algorithm

Node	A	B	C	D	E	F
Disjoint Set	A	B	C	D	E	F



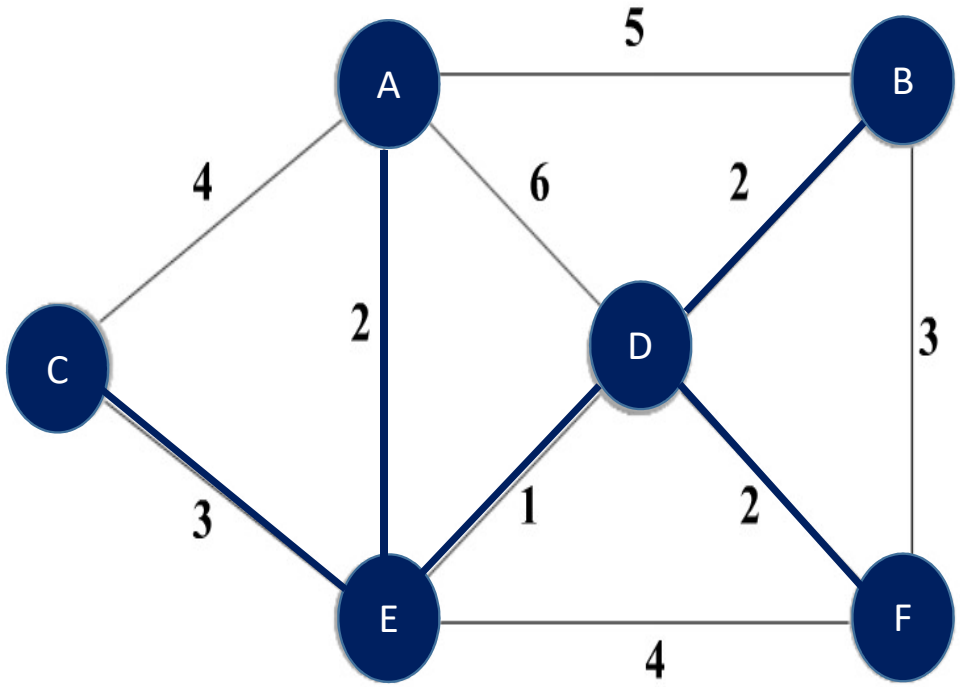
Prim's Algorithm

Node	A	B	C	D	E	F
Disjoint Set	A	B	C	D	E	F



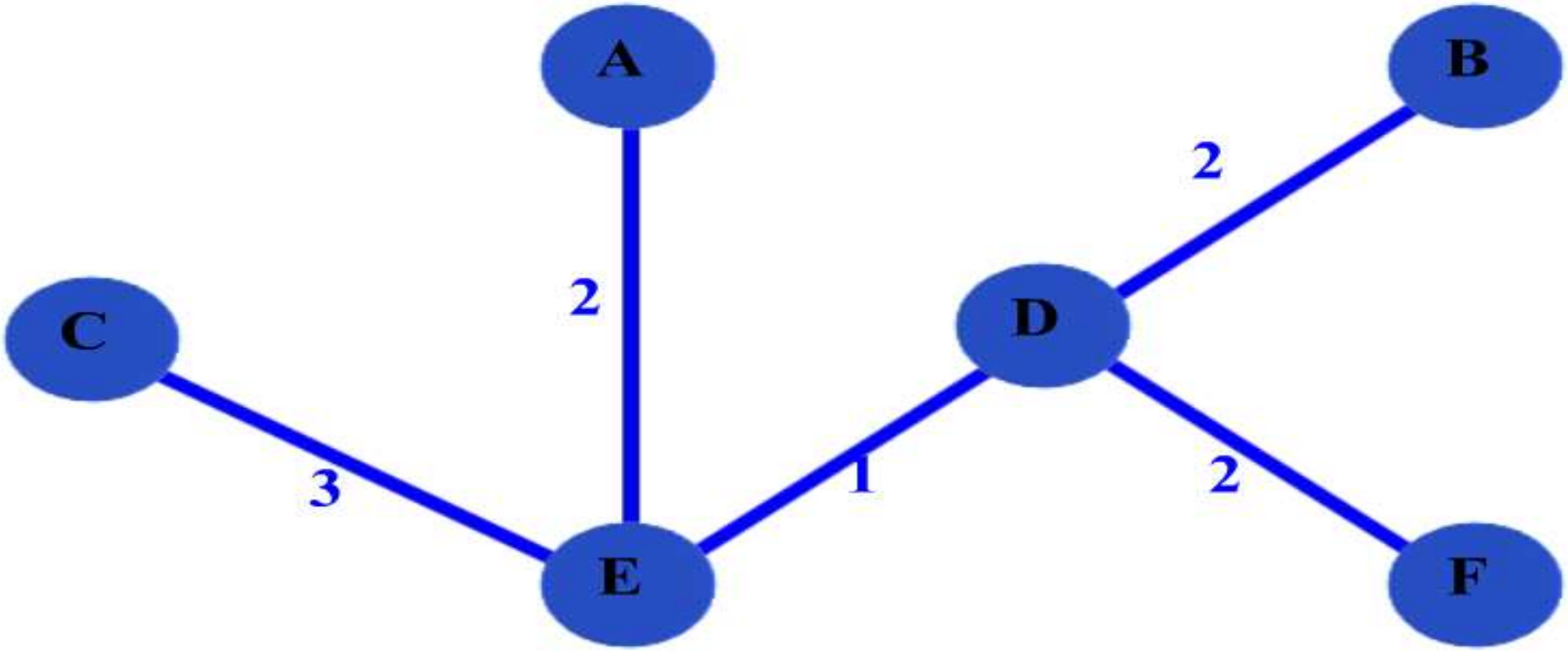
Prim's Algorithm

Node	A	B	C	D	E	F
Disjoint Set	A	B	C	D	E	F



Prim's Algorithm

minimum- spanning tree



Prim's Algorithm

```
T = ∅;
U = { 1 };
while (U ≠ V)
    let (u, v) be the lowest cost edge such that u ∈ U and v ∈ V - U;
    T = T ∪ {(u, v)}
    U = U ∪ {v}
```

Prim's Algorithm (Complexity)

- Need priority queue for locating the nearest vertex
  - Use unordered array to store the priority queue:

    Efficiency: $\Theta(n^2)$

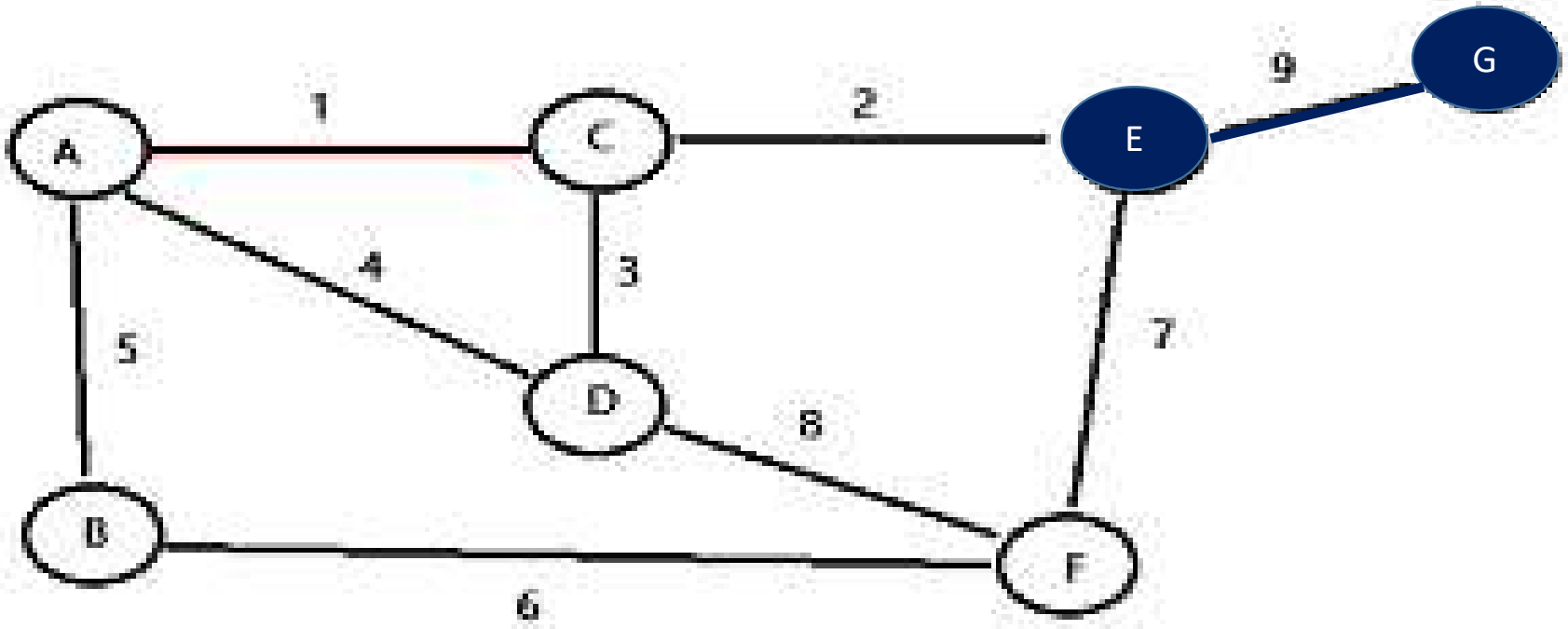
  - Use binary *min-heap* to store the priority queue

    Efficiency: For graph with n vertices and m edges:

    $O(m \log n)$

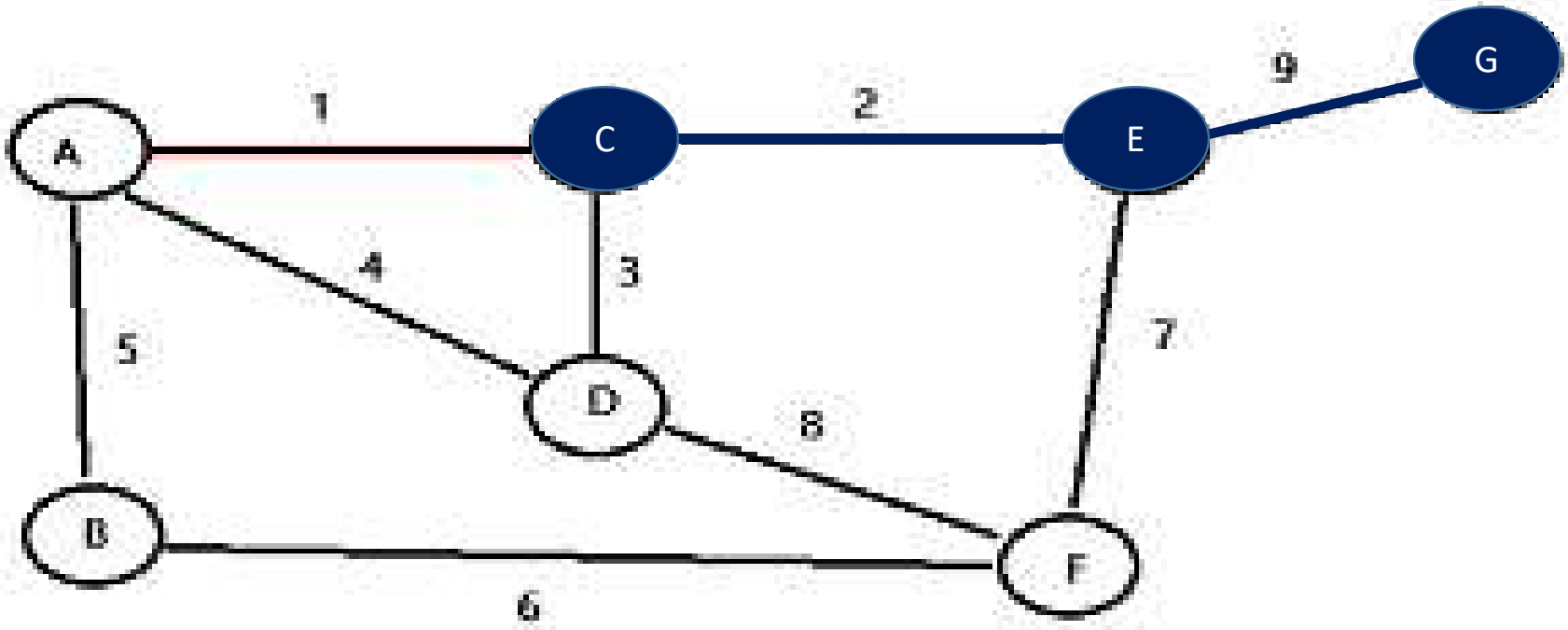
EXAMPLE – 2 (PRIM'S ALGORITHM)

Node	A	B	C	D	E	F	G
Disjoint Set	A	B	C	D	E	F	G



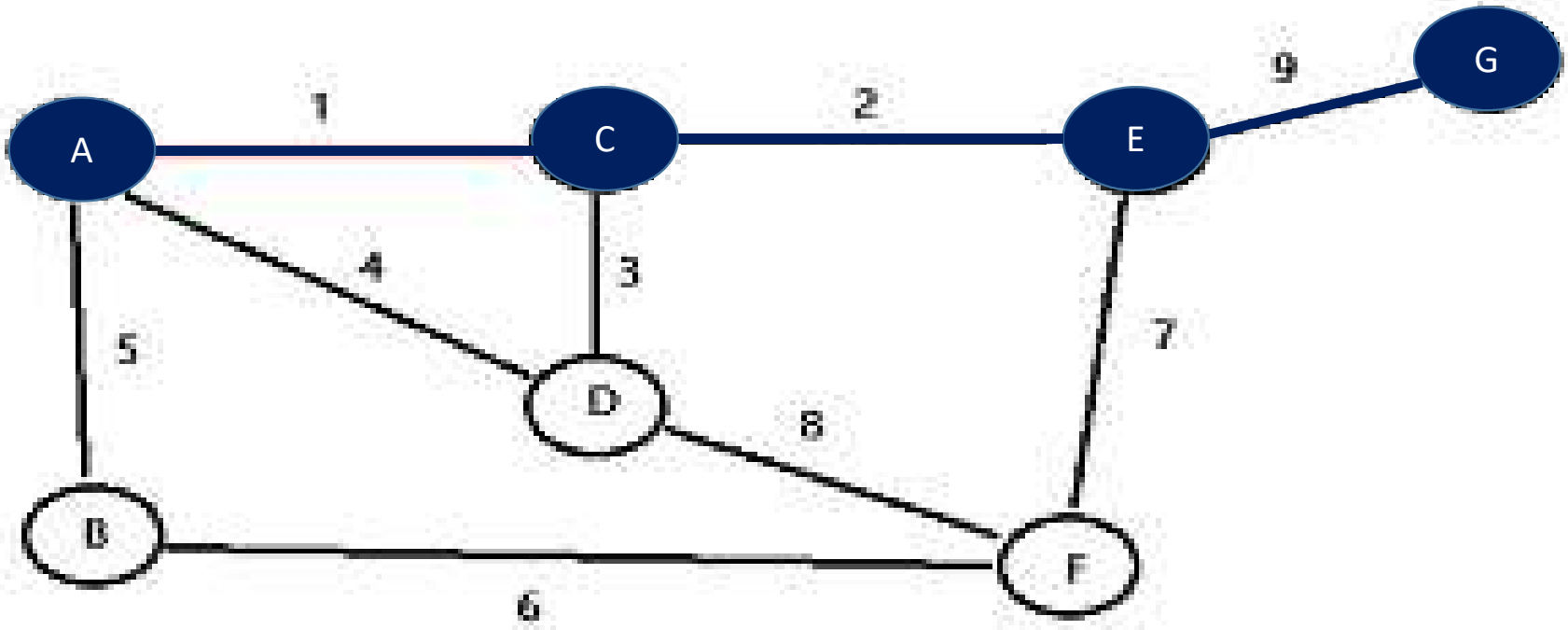
EXAMPLE – 2 (PRIM'S ALGORITHM)

Node	A	B	C	D	E	F	G
Disjoint Set	A	B	C	D	E	F	G



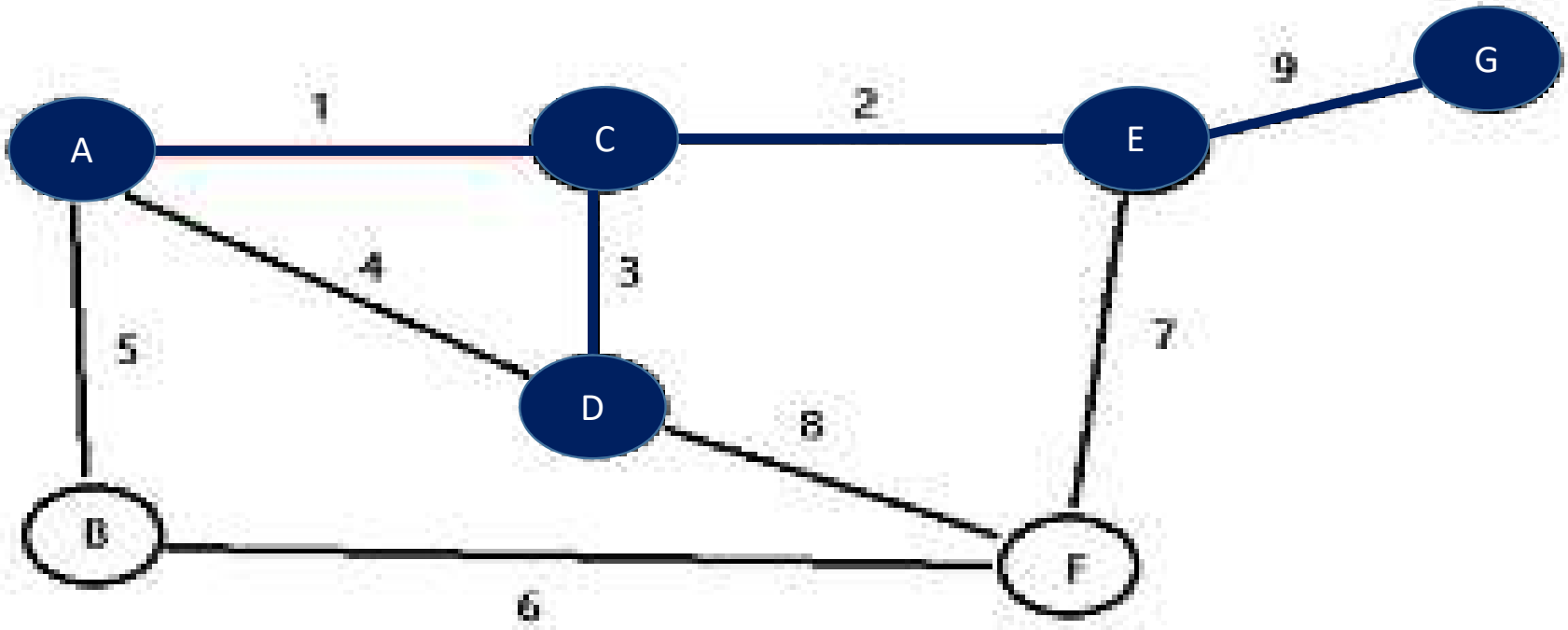
EXAMPLE – 2 (PRIM'S ALGORITHM)

Node	A	B	C	D	E	F	G
Disjoint Set	A	B	C	D	E	F	G



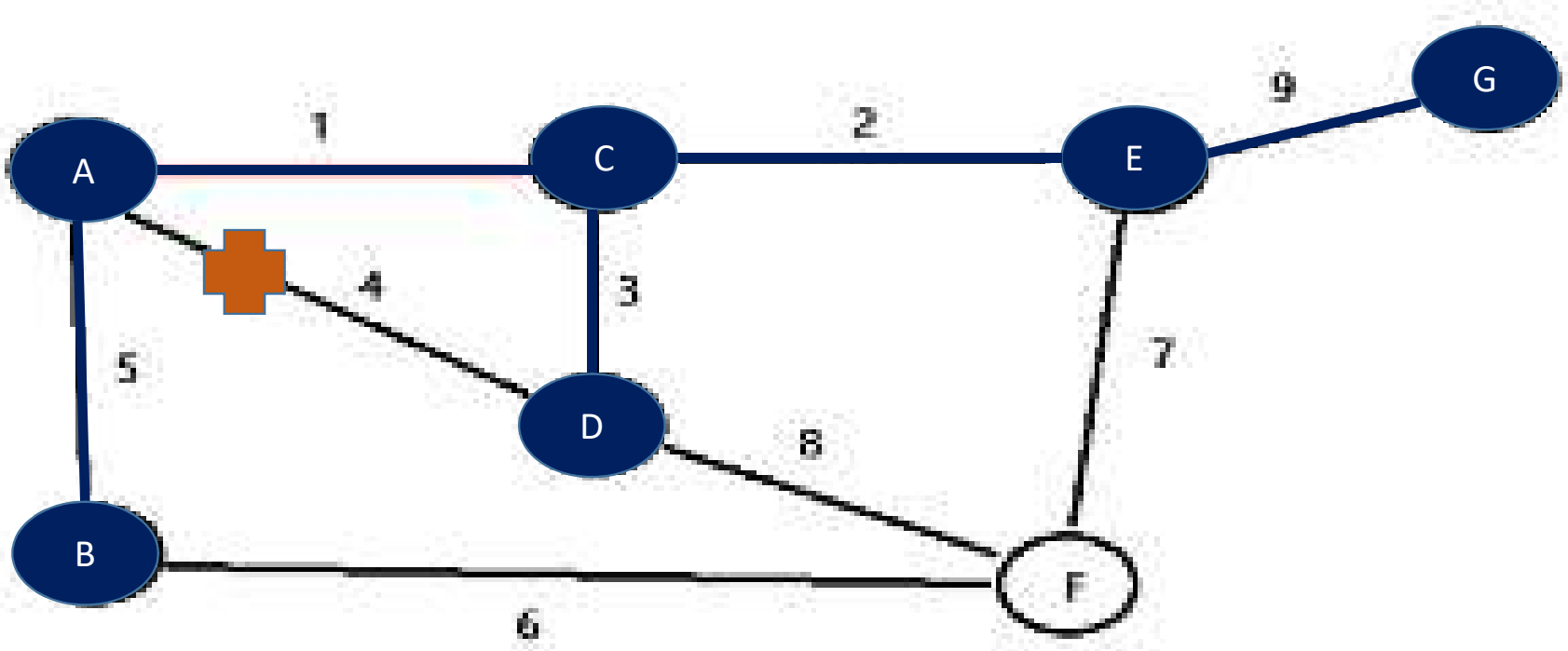
EXAMPLE – 2 (PRIM'S ALGORITHM)

Node	A	B	C	D	E	F	G
Disjoint Set	A	B	C	D	E	F	G



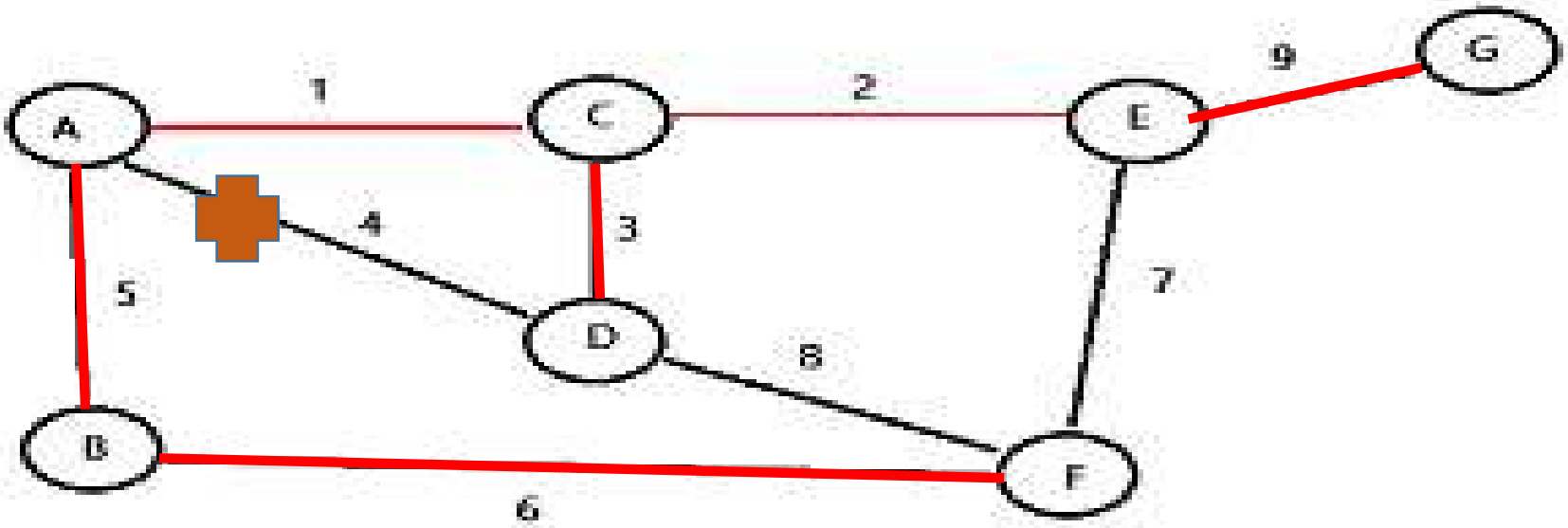
EXAMPLE – 2 (PRIM'S ALGORITHM)

Node	A	B	C	D	E	F	G
Disjoint Set	A	B	C	D	E	F	G



EXAMPLE – 2 (PRIM'S ALGORITHM)

Node	A	B	C	D	E	F	G
Disjoint Set	A	B	C	D	E	F	G

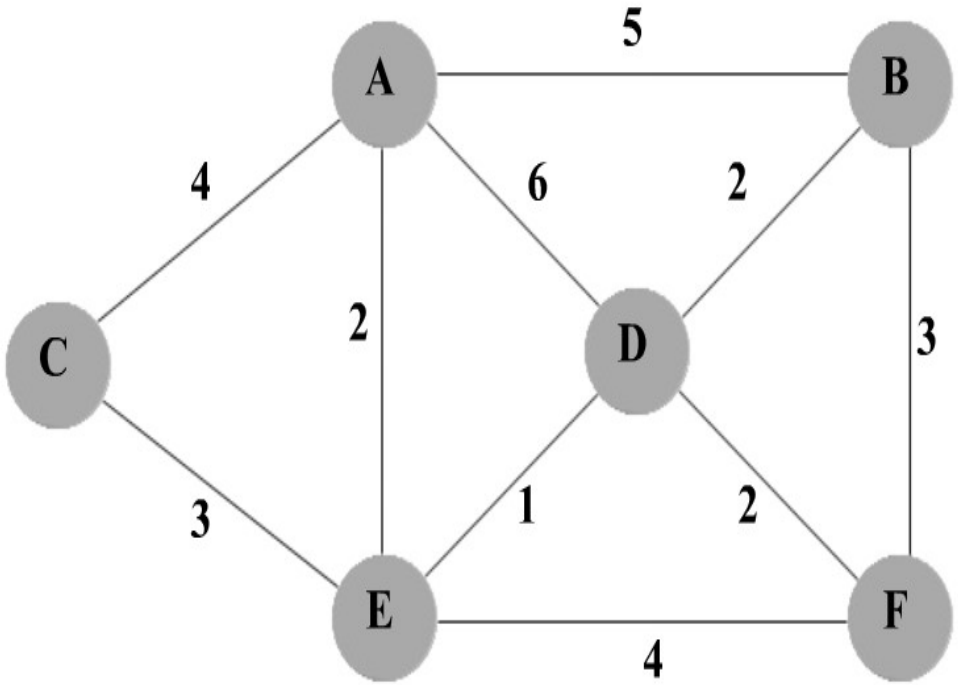


Krushkal's Algorithm

```
MST-KRUSKAL(G, w)
1   A ← ∅
2   for each vertex v ∈ V[G]
3       do MAKE-SET(v)
4   sort the edges of E into nondecreasing order by weight w
5   for each edge (u, v) ∈ E, taken in nondecreasing order by weight
6       do if FIND-SET(u) ≠ FIND-SET(v)
7             then A ← A ∪ {(u, v)}
8                  UNION(u, v)
9   return A
```

Krushkal's Algorithm

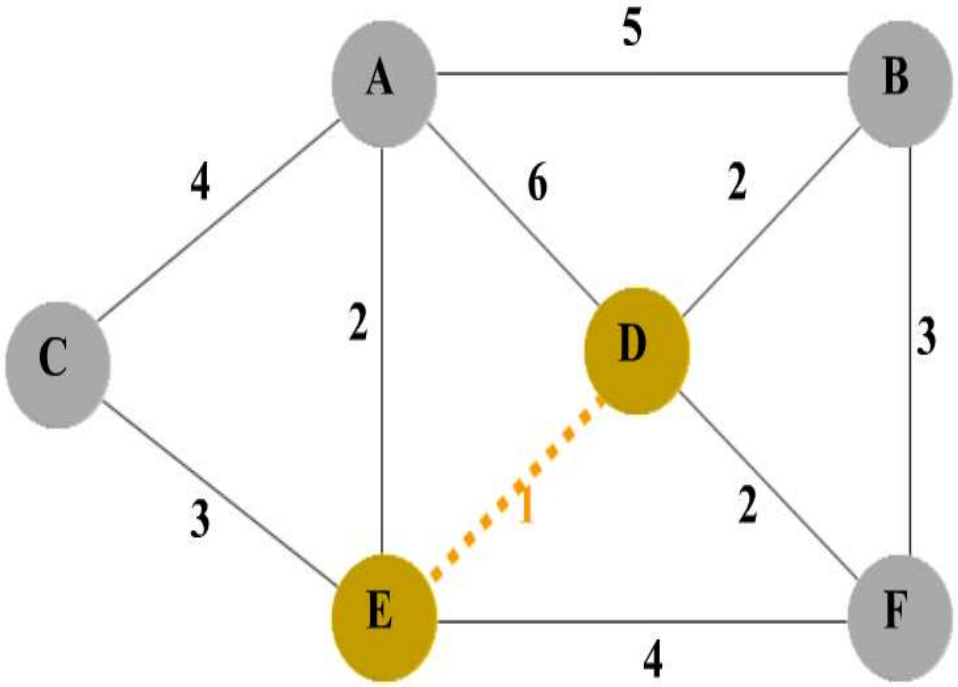
Node	A	B	C	D	E	F
Disjoint Set	A	B	C	D	E	F



Kruskal's Algorithm

Krushkal's Algorithm

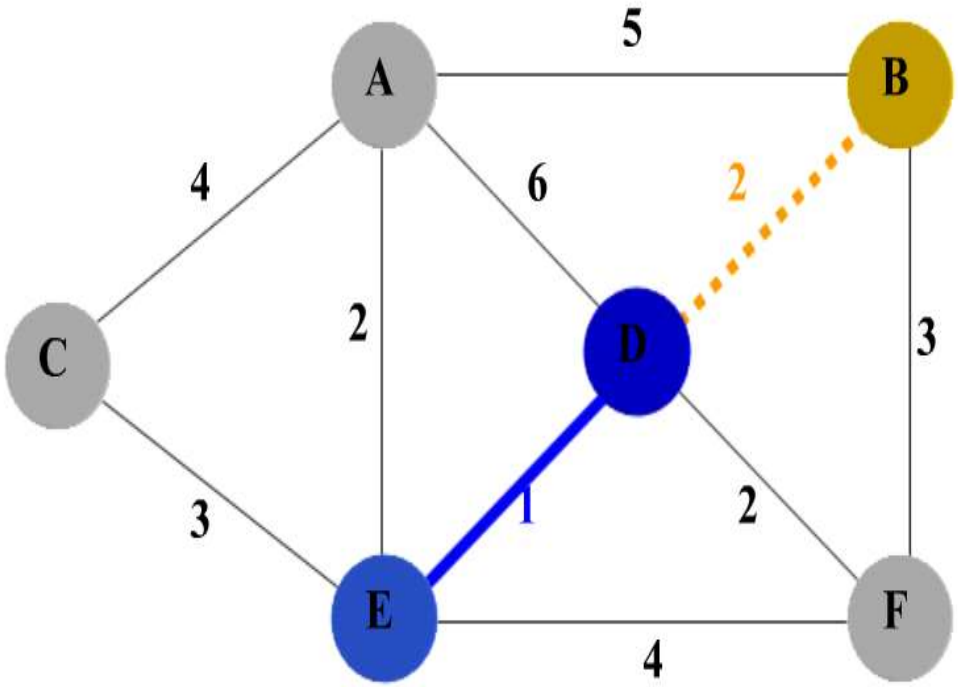
Node	A	B	C	D	E	F
Disjoint Set	A	B	C	D	D	F



Kruskal's Algorithm

Krushkal's Algorithm

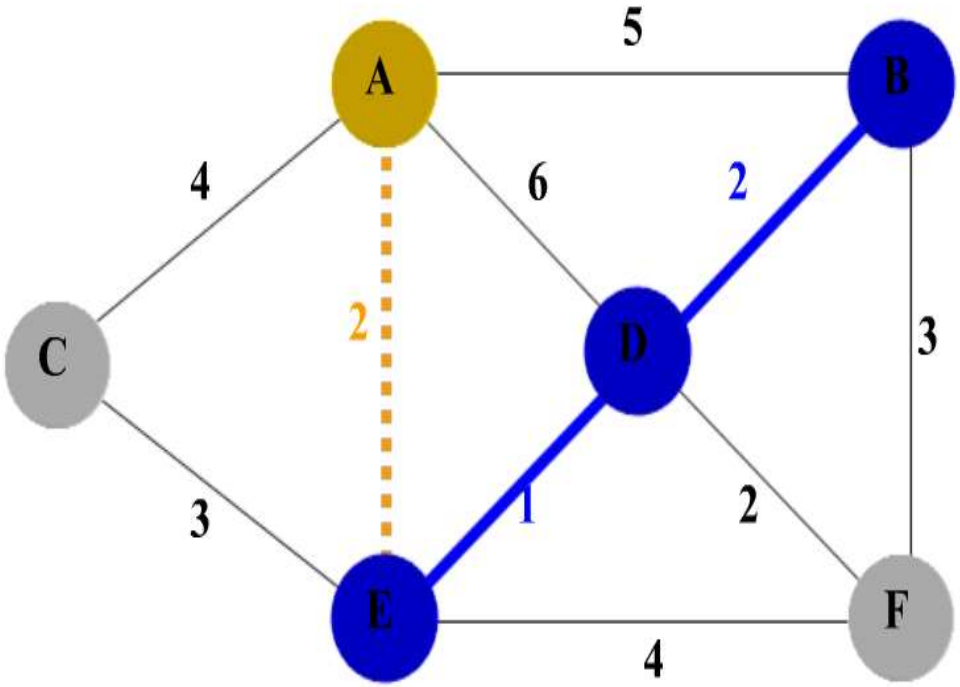
Node	A	B	C	D	E	F
Disjoint Set	A	B	C	B	B	F



Kruskal's Algorithm

Krushkal's Algorithm

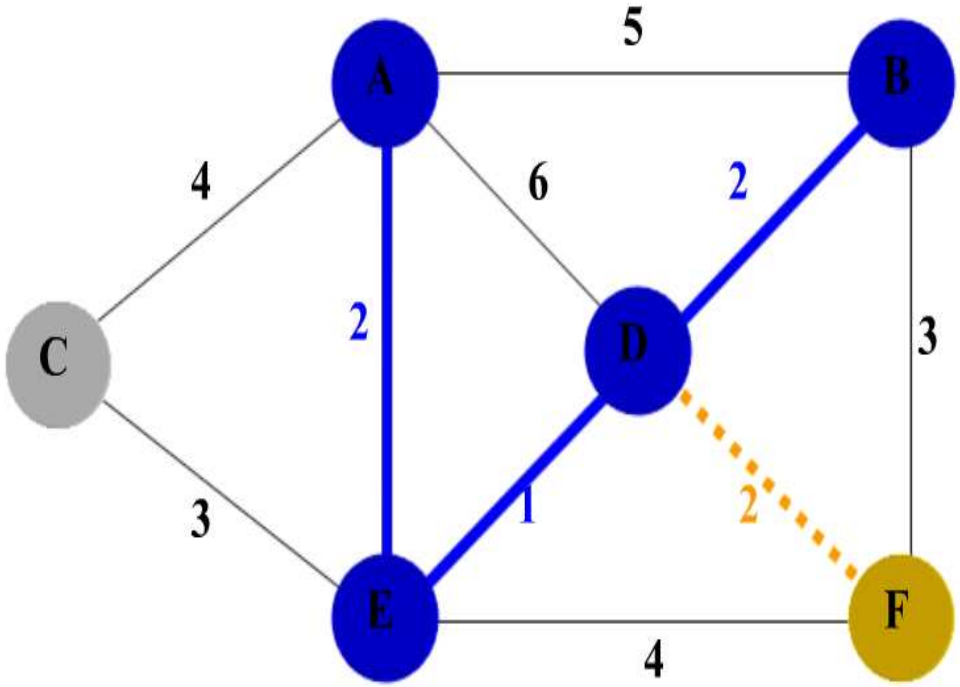
Node	A	B	C	D	E	F
Disjoint Set	A	A	C	A	A	F



Kruskal's Algorithm

Krushkal's Algorithm

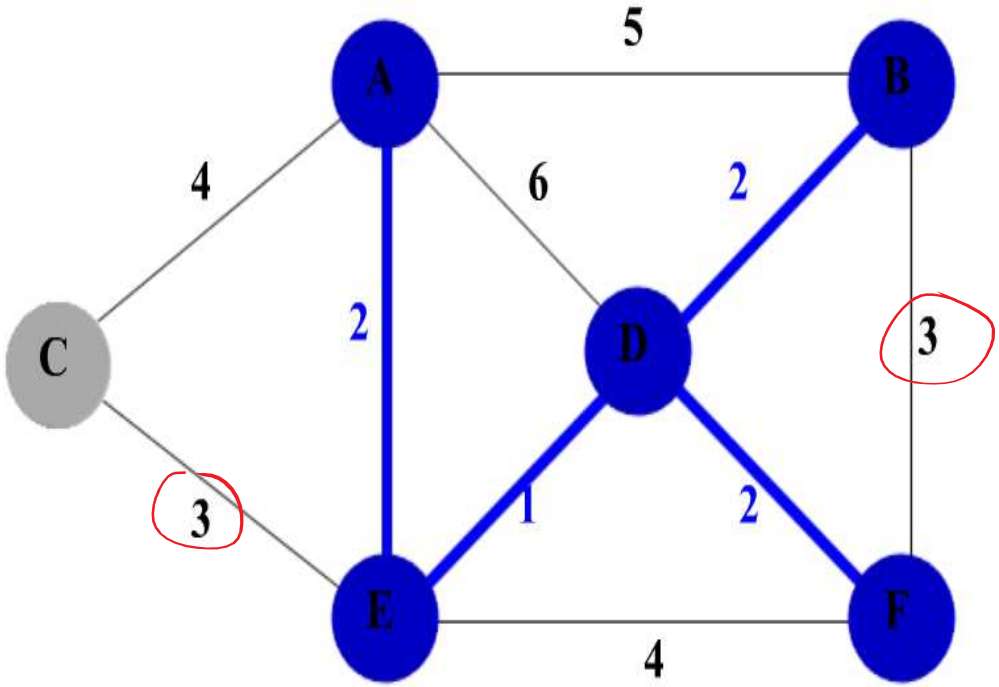
Node	A	B	C	D	E	F
Disjoint Set	A	A	C	A	A	F



Kruskal's Algorithm

Krushkal's Algorithm

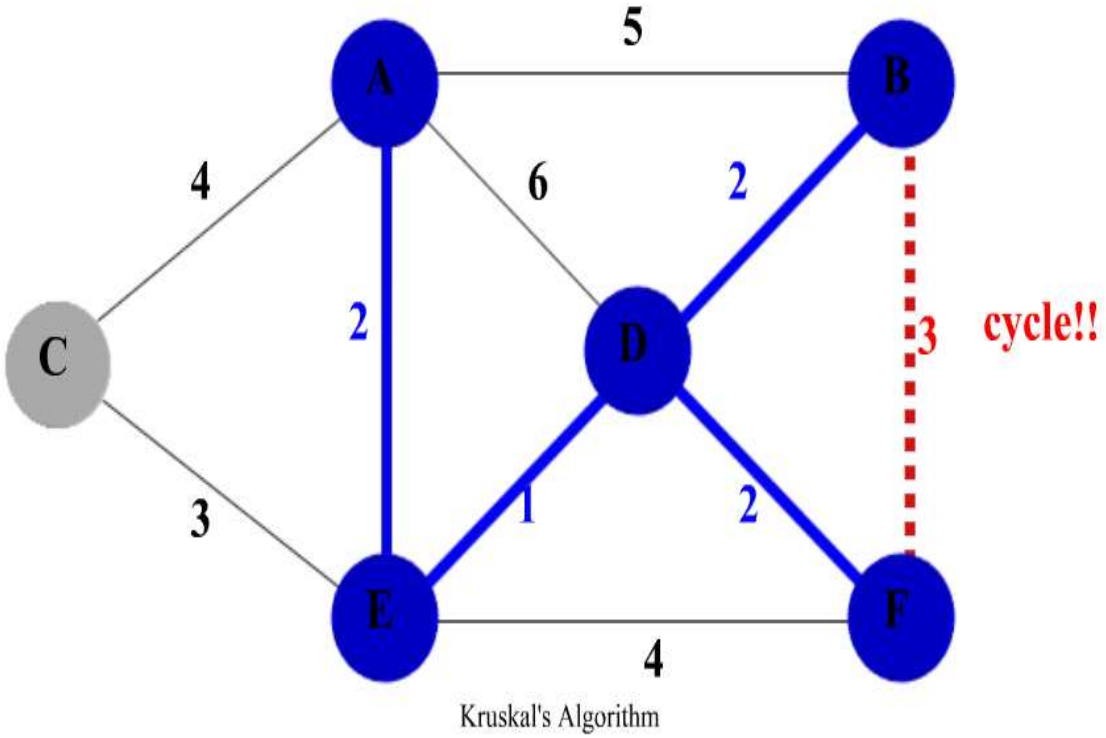
Node	A	B	C	D	E	F
Disjoint Set	A	A	C	A	A	A



Kruskal's Algorithm

Krushkal's Algorithm

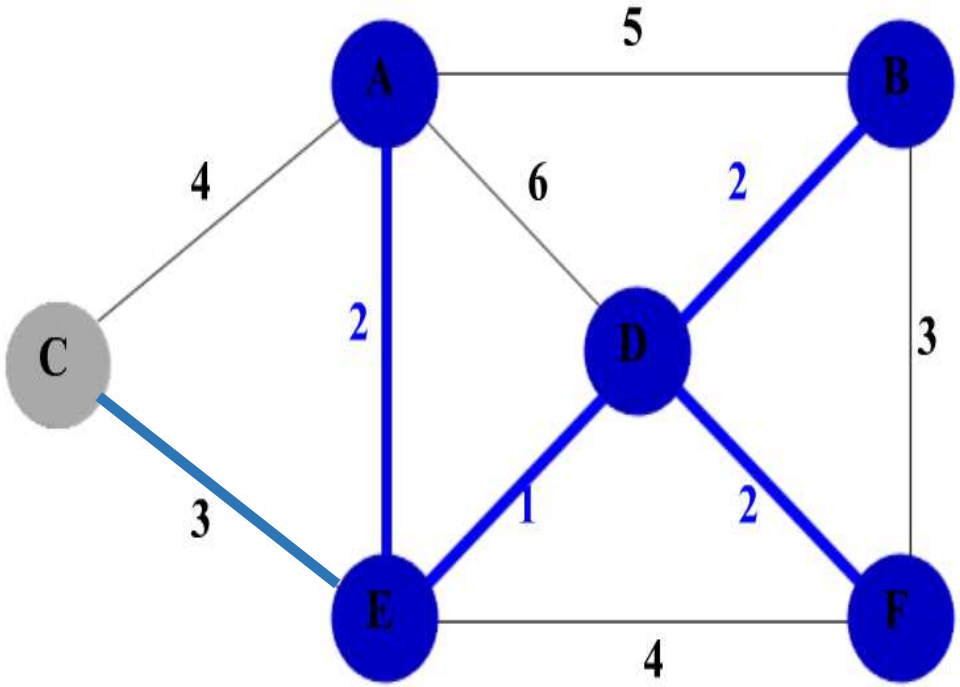
Node	A	B	C	D	E	F
Disjoint Set	A	A	C	A	A	A



Kruskal's Algorithm

Krushkal's Algorithm

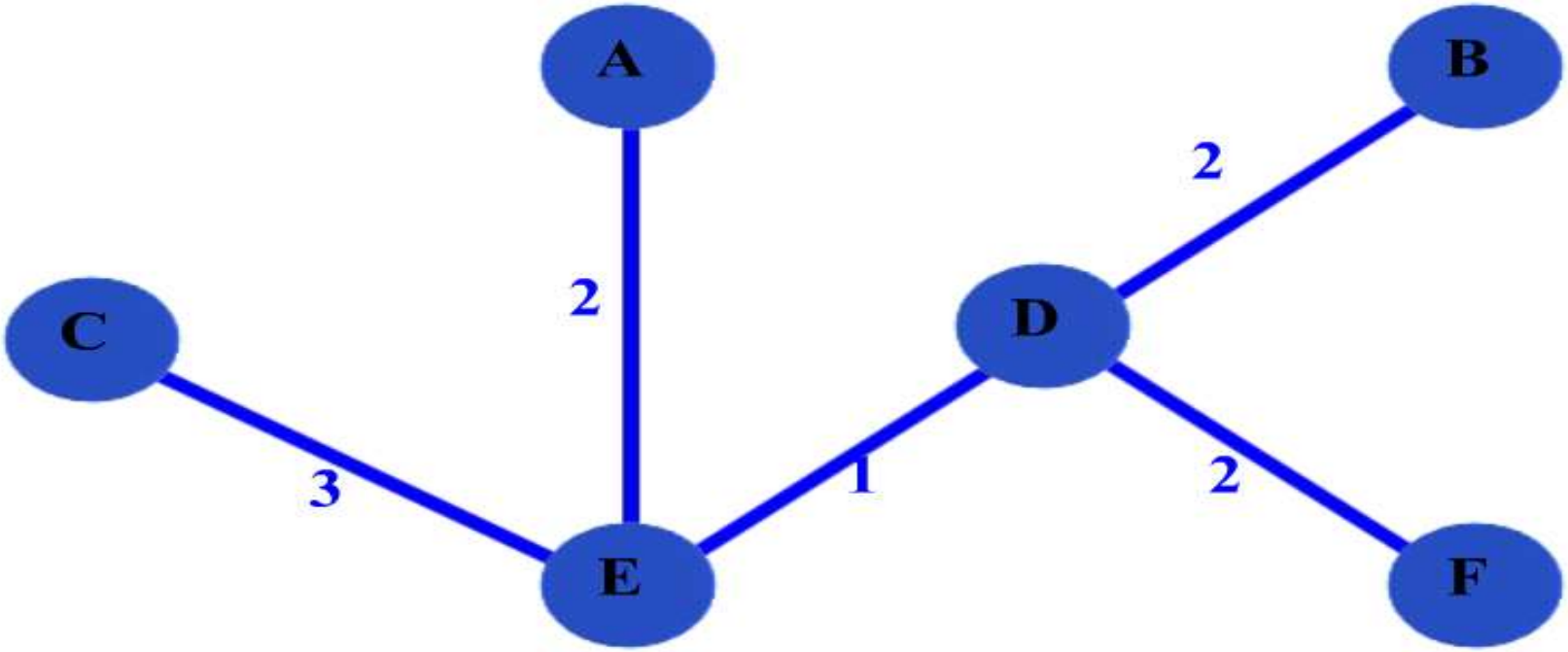
Node	A	B	C	D	E	F
Disjoint Set	A	A	A	A	A	A



Kruskal's Algorithm

Krushkal's Algorithm

minimum- spanning tree $T(W) = 10$



Kruskal's Algorithm

Krushkal's Algorithm

```
MST-KRUSKAL(G, w)
1   A ← ∅
2   for each vertex v ∈ V[G]
3       do MAKE-SET(v)
4   sort the edges of E into nondecreasing order by weight w
5   for each edge (u, v) ∈ E, taken in nondecreasing order by weight
6       do if FIND-SET(u) ≠ FIND-SET(v)
7             then A ← A ∪ {(u, v)}
8                  UNION(u, v)
9   return A
```

Krushkal's Algorithm (Complexity)

1. $O(EV)$ when disjoint-set data structure are not used.
2. When use disjoint-set data structure with union-by-rank and path-compression heuristics:
   a) Initializing the set A in line 1 takes $O(1)$ time.
   b) $O(V)$ MAKE-SET operations in lines 2-3
   c) Sort the edges in line 4 takes $O(E \log E)$ time.
   d) The for loop of lines 5-8 performs $O(E)$ FIND-SET and UNION operations on the disjoint-set forest
      - b) and d) take a total of $O((V+E)\alpha(V))$
      - Line 9: $O(V+E)$

      Note that: $E \geq V\text{-}1$ and $\alpha(V) = O(\log V) = O(\log E)$ and $E \leq V^2$

      SO total time is: $O(E \log V)$

Applications of Minimum-Cost Spanning Trees

Minimum-cost spanning trees have many applications. Some are:

- Building cable networks OR road network that join n locations with minimum cost.
- In Image Processing also MST can be used for Image Segmentation
- In pattern recognition minimum spanning trees can be used to find noisy pixels.